



TOP 20

LANGCHAIN

CONCEPTS

Before You Build with **LangChain**,
Learn These 20 Concepts



Chandra Sekhar

AI Engineer & Educator



What it is

A **Chat Model** is an LLM that works with **messages**, not plain text.

Examples

- GPT-4, GPT-4o, Claude, Gemini
- Accepts: System, Human, AI messages
- Returns: AI message

Why LangChain uses Chat Models

Because real apps are **conversational**, not single prompts.

PYTHON CODE

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-4o-mini", temperature=0)

response = model.invoke("Explain LangChain in one sentence")
print(response.content)
```



What they are

Messages structure **who is saying what**.

Type	Purpose
System	Sets behavior/rules
Human	User input
AI	Model response

PYTHON CODE

```
from langchain_core.messages import SystemMessage,
HumanMessage

messages = [
    SystemMessage(content="You are a helpful Python tutor"),
    HumanMessage(content="Explain lists in Python")
]

response = model.invoke(messages)
print(response.content)
```



What it is

Chat history is the **conversation memory** — previous messages sent to the model.

- **Without history** → model forgets context
- **With history** → model understands continuity

⚠ *Chat history ≠ Memory (memory is managed, summarized, stored)*

PYTHON CODE

```
chat_history = [  
    HumanMessage(content="What is LangChain?"),  
    AIMessage(content="LangChain is a framework for LLM  
apps."),  
    HumanMessage(content="Give an example")  
]  
  
response = model.invoke(chat_history)
```



What it is

A **tool** is a function the LLM can call to perform actions beyond text generation.

Why tools matter

LLMs **can think**, but **tools let them act**. Tools enable:

- Real-time data access (web search, APIs)
- Calculations and database queries
- Custom business logic execution

Examples

Calculator, Web search, Database queries, Custom Python functions

PYTHON CODE

```
from langchain.tools import tool

@tool
def multiply(a: int, b: int) → int:
    """Multiply two numbers."""
    return a * b
```



What it is

Force the model to return **structured data** (JSON, Pydantic models) instead of free-form text.

Why use it?

- **Reliable:** Guaranteed format every time
- **Machine-readable:** Easy to parse and process
- **Type-safe:** Validates data structure automatically
- No regex parsing needed

Use cases

APIs, data extraction, form filling, structured analysis

PYTHON CODE

```
from pydantic import BaseModel
from langchain.output_parsers import PydanticOutputParser

class Answer(BaseModel):
    summary: str
    confidence: float
    tags: list[str]

parser = PydanticOutputParser(pydantic_object=Answer)
```



What it is

Reusable prompts with **placeholders** for dynamic input injection. Instead of hardcoding prompts, use templates with variables.

Why use templates?

- Reuse prompts across different inputs
- Maintain consistency in prompt structure
- Easy to update and version control

PYTHON CODE

```
from langchain.prompts import ChatPromptTemplate

# Create template with placeholder
prompt = ChatPromptTemplate.from_template(
    "Explain {topic} to a {audience}"
)

# Fill in values dynamically
formatted = prompt.invoke({
    "topic": "LangChain",
    "audience": "beginner"
})
```



What it is

A **chain** is a pipeline that combines multiple components:

```
Input → Prompt → Model → Output
```

Chains connect prompts, models, and parsers into reusable workflows.

LCEL (LangChain Expression Language)

Modern syntax using `|` operator for easy composition. Makes chains streamable and parallelizable.

PYTHON CODE

```
from langchain_core.output_parsers import StrOutputParser

# LCEL chain composition
chain = prompt | model | StrOutputParser()

# Execute the chain
result = chain.invoke({"topic": "RAG"})
print(result)
```



What they are

Load data from various sources into LangChain **Document** objects.

Supported formats

PDFs, Text files, Websites, CSVs, JSON, Word docs, and more

```
Each doc = Document(page_content, metadata)
```

PYTHON CODE

```
from langchain.document_loaders import TextLoader,
PyPDFLoader

# Load text file
loader = TextLoader("notes.txt")
docs = loader.load()

# Load PDF
pdf_loader = PyPDFLoader("document.pdf")
pdf_docs = pdf_loader.load()
```



What it is

Instead of guessing, the model retrieves relevant documents and uses them as context.

Setup Flow

Docs → Split → Embed → Store

Documents are chunked, converted to vectors, and stored in vector DB.

Query Flow

Query → Retrieve → Augment → Generate

Query finds similar docs, adds them to prompt context, then generates answer.



Why chunking?

LLMs have **context limits**. Large documents must be split into smaller, manageable chunks.

Key parameters

- **chunk_size**: Max characters per chunk
- **chunk_overlap**: Overlap between chunks (preserves context)

PYTHON CODE

```
from langchain.text_splitter import  
RecursiveCharacterTextSplitter  
  
splitter = RecursiveCharacterTextSplitter(  
    chunk_size=500,  
    chunk_overlap=50 # Preserves context between chunks  
)  
  
chunks = splitter.split_documents(docs)
```



What they are

Convert text into **numerical vectors** (arrays of numbers) that capture semantic meaning.

Why embeddings?

Enable **similarity search** - find documents with similar meaning, not just exact matches.

PYTHON CODE

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings()

# Convert text to vector
vector = embeddings.embed_query("What is LangChain?")
```



What they are

Specialized databases optimized for storing and querying **embeddings** efficiently.

Popular options

- **FAISS:** Local, fast, open-source
- **Chroma:** Easy to use, embedded
- **Pinecone:** Managed, scalable cloud

PYTHON CODE

```
from langchain.vectorstores import FAISS

# Store document embeddings
vectorstore = FAISS.from_documents(chunks, embeddings)

# Save for later use
vectorstore.save_local("my_index")
```



What they are

Retrievers fetch the **most relevant documents** from a vector store based on a query.

How they work

Convert query to embedding → Find similar vectors → Return top-k documents

PYTHON CODE

```
# Create retriever from vectorstore
retriever = vectorstore.as_retriever(
    search_kwargs={"k": 3} # Return top 3 results
)

# Retrieve relevant docs
docs = retriever.invoke("What is LangChain?")
```



RAG Architecture

UI → API → Chain → Retriever → LLM → Output

User query flows through retrieval, then generation.

Agent Architecture

User → Agent → Tools → Memory → LLM

Agent orchestrates tools and maintains conversation state.

PYTHON CODE

```
# RAG Flow:  
# Query → Retrieve docs → Augment prompt → Generate  
  
# Agent Flow:  
# Query → Agent decides → Call tools → Update state → Respond
```



What LangChain integrates with

LangChain provides unified interfaces for:

- **LLMs:** OpenAI, Anthropic, Gemini, Cohere
- **Vector DBs:** Pinecone, Chroma, FAISS, Weaviate
- **Tools:** Web search, APIs, databases

Benefit

Switch providers without changing your code structure.

PYTHON CODE

```
from langchain_anthropic import ChatAnthropic
from langchain_openai import ChatOpenAI

# Easy to switch models
claude = ChatAnthropic(model="claude-3")
gpt = ChatOpenAI(model="gpt-4o")
```



What it is

Using **different models for different tasks** to optimize cost and performance.

Strategy

- **Fast/cheap model:** Simple tasks, retrieval, filtering
- **Powerful model:** Complex reasoning, final answers
- Reduces costs while maintaining quality

PYTHON CODE

```
from langchain_openai import ChatOpenAI

# Use cheap model for simple tasks
fast_model = ChatOpenAI(model="gpt-4o-mini", temperature=0)

# Use powerful model for reasoning
smart_model = ChatOpenAI(model="gpt-4o", temperature=0.7)
```



What they are

Agents are **autonomous systems** that decide what actions to take using tools.

Agent loop

They work in cycles:

Thought → Action → Observation → Repeat

Agent analyzes, chooses tool, observes result, decides next step.

PYTHON CODE

```
from langchain.agents import create_openai_tools_agent

# Agent decides which tools to use
agent = create_openai_tools_agent(
    llm=model,
    tools=[calculator, search, db_query],
    prompt=prompt
)
```



What it is

Everything in LangChain implements the **Runnable** interface: Prompts, Models, Parsers, Retrievers.

Key benefits

- **Composable:** Chain with `|` operator
- **Streamable:** Get results as they generate
- **Parallelizable:** Run multiple chains simultaneously

PYTHON CODE

```
# Everything is a Runnable
chain = prompt | model | parser

# Stream results
for chunk in chain.stream({"input": " ... "}):
    print(chunk)

# Batch processing
results = chain.batch([{"input": " ... "}]])
```



What it is

Define **custom Python functions** that agents can call. The agent decides when and how to use them.

How it works

Use `@tool` decorator. Agent reads function docstring to understand when to call it.

PYTHON CODE

```
from langchain.tools import tool

@tool
def get_weather(city: str) → str:
    """Get current weather for a city."""
    # Your custom logic here
    return f"Weather in {city} is sunny"

# Agent will call this when user asks about weather
```



What it is

LangGraph = **stateful, multi-step, cyclic workflows** for complex agent systems.

Better than chains for agents with loops, conditional flows, and multi-agent coordination.

Key concepts

- **Nodes:** Individual steps/functions
- **Edges:** Control flow between nodes
- **State:** Shared data across the graph

PYTHON CODE

```
from langgraph.graph import StateGraph, START, END

graph = StateGraph(dict)
graph.add_node("retrieve", retrieve_docs)
graph.add_node("generate", generate_answer)
graph.add_edge(START, "retrieve")
graph.add_edge("retrieve", "generate")
graph.add_edge("generate", END)
```



Want more content like this?



Tap that follow button and
stay in the loop!



Like



Comment



Share



Save